

https://moodle.joinville.udesc.br/pluginfile.php/429003/mod_resource/content/3/PAA_03.pdf

Projeto e Análise de Algoritmos

Tempo de Algoritmos Recursivos

André Tavares da Silva
andre.silva@udesc.br

Algoritmos Recursivos

- Na maioria linguagens de programação atuais, uma função pode chamar a si mesma. Uma função assim é chamada função recursiva.
- Existem muitos problemas que são definidos com base neles mesmos, ou seja, podem ser descritos por instâncias do próprio problema. Para estas classes de problemas, o conceito de recursividade torna-se muito útil. Ex.: $n! = n * (n-1)!$

Algoritmos Recursivos

- Deve-se tomar alguns cuidados ao se fazer funções recursivas.
- Um cuidado importante a se verificar, é a existência de um **critério de parada** para determinar quando a função deverá parar de chamar a si mesma. Este cuidado serve para impedir que a função se chame infinitas vezes.

Algoritmos Recursivos

- No caso da função fatorial, podemos escrevê-la da seguinte forma:

```
int fatorial(int n) {  
    return n * fatorial(n-1);  
}
```

- Qual o critério de parada?

Algoritmos Recursivos

- No caso da função fatorial, podemos escrevê-la da seguinte forma:

```
int fatorial(int n) {  
    if(n<=1)  
        return 1;  
    return n * fatorial(n-1);  
}
```

Algoritmos Recursivos

- Geralmente a análise de complexidade de algoritmos recursivos é um pouco mais complexa pois é preciso entender bem a recursividade do algoritmo.
- Quantas vezes um algoritmo chama a si mesmo?
- Existe um pior caso? Quantas chamadas recursivas são feitas?
- Devemos considerar o tempo gasto na chamada de função? E o espaço utilizado na memória?

Exemplo (Fatorial)

```
int fatorial( int n ){  
    if (n <= 1)  
        return 1;  
    return n * fatorial( n-1 );  
}
```

Exemplo (Fatorial)

```
int fatorial( int n ){  
    if (n <= 1)  
        return 1;  
    return n * fatorial( n-1 );  
}
```

Relação de Recorrência:

$$T(n) = T(n-1) + O(1)$$

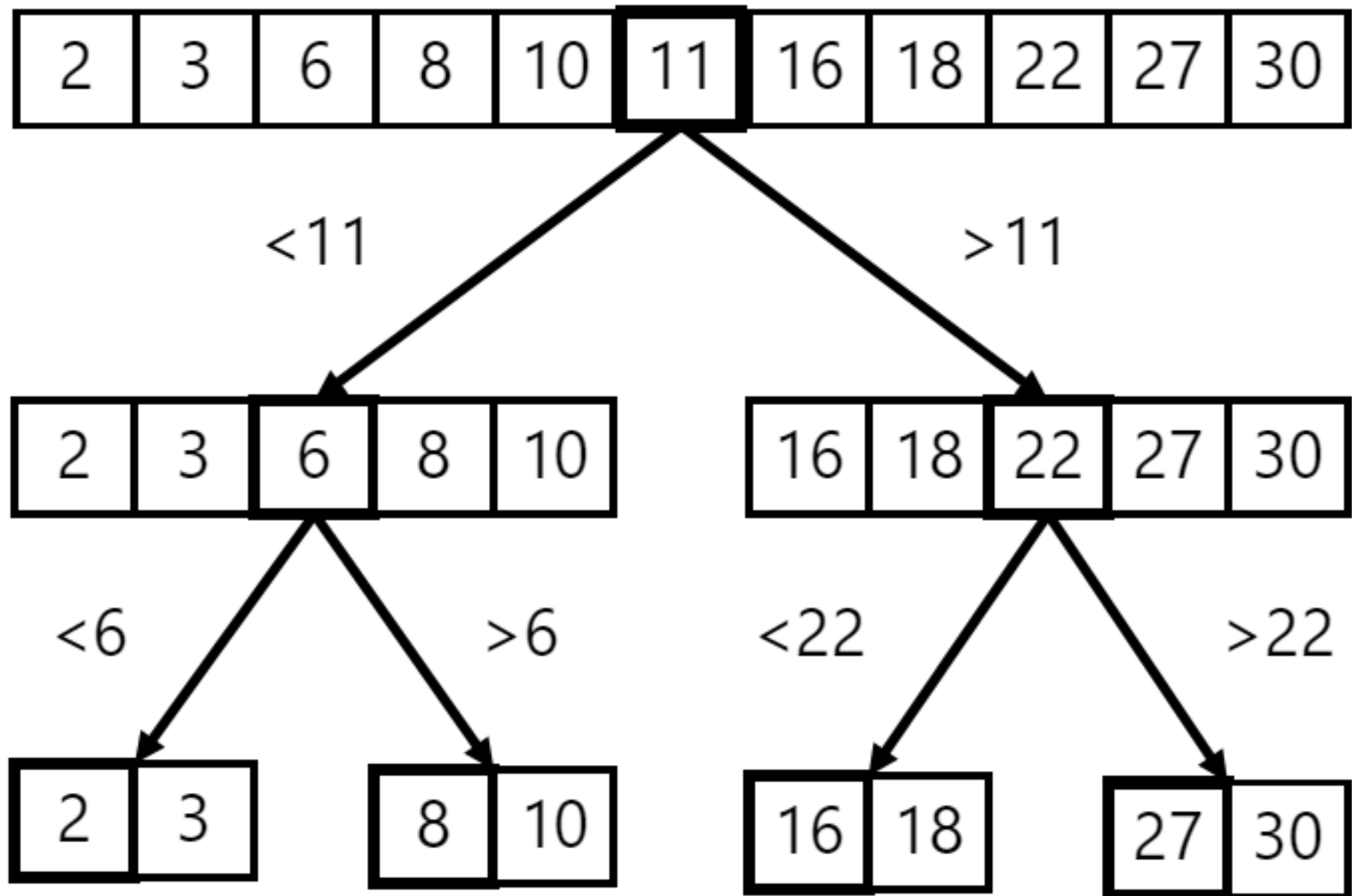
$$T(1) = O(1)$$

Exemplo (Pesquisa Binária)

```
int pesqbin(int *v, int p, int r, int e){  
    int q;  
    if ( r < p )  
        return -1;  
    q = (p + r)/2;  
    if (e == v[q])  
        return q;  
    if (e < v[q])  
        return pesqbin(v, p, q-1, e);  
    else  
        return pesqbin(v, q+1, r, e);  
}
```

** Lembrete: o vetor deve estar ordenado!!*

Binary Search



Exemplo (Pesquisa Binária)

```
int pesqbin(int *v, int p, int r, int e){  
    int q;  
    if ( r < p )  
        return -1;  
    q = (p + r)/2;  
    if (e == v[q])  
        return q;  
    if (e < v[q])  
        return pesqbin(v, p, q-1, e);  
    return pesqbin(v, q+1, r, e);  
}
```

Relação de Recorrência:

$$T(n) = T(n/2) + O(1)$$

$$T(1) = O(1)$$

A relação abaixo descreve de forma mais precisa a execução, mas a simplificação acima não altera a complexidade:

$$T(n) = T(n/2 - 1) + O(1)$$

$$T(1) = O(1)$$

Relação de Recorrência

$$T(n) = T(n/2) + 1$$

$$T(n/2) = T(n/2^2) + 1$$

$$T(n/2^2) = T(n/2^3) + 1$$

...

...

$$T(n/2^{h-1}) = T(n/2^h) + 1$$

Relação de Recorrência:

$$T(n) = T(n/2) + O(1)$$

$$T(1) = O(1)$$

Relação de Recorrência

$$T(n) = T(\cancel{n/2}) + 1$$

$$\cancel{T(n/2)} = T(\cancel{n/2^2}) + 1$$

$$\cancel{T(n/2^2)} = T(\cancel{n/2^3}) + 1$$

...

$$\cancel{T(n/2^{h-1})} = T(n/2^h) + 1$$

Relação de Recorrência:

$$T(n) = T(n/2) + O(1)$$

$$T(1) = O(1)$$

$$T(n) = \underbrace{1 + 1 + \dots + 1}_{h \text{ vezes}} + T(1)$$

h vezes => ops... Mas quanto vale h??

Relação de Recorrência

$$T(n) = T(\cancel{n/2}) + 1$$

$$\cancel{T(n/2)} = T(\cancel{n/2^2}) + 1$$

$$\cancel{T(n/2^2)} = T(\cancel{n/2^3}) + 1$$

...

$$\cancel{T(n/2^{h-1})} = T(n/2^h) + 1$$

$$O(\log n)$$

$$\frac{n}{2^h} = 1$$

$$\begin{aligned} n &= 2^h \\ h &= \log_2 n \end{aligned}$$

Mudança de base:

$$\log_b n = \frac{\log_a n}{\log_a b}$$

$$T(n) = \underbrace{1 + 1 + \dots + 1}_{h \text{ vezes}} + T(1)$$

h vezes, ou seja, $\log_2 n$ vezes

Exemplo (Pesquisa Binária)

```
int pesqbin2 (int *v, int n, int e) {  
    int p, q, r;  
  
    p = 0; r = n-1;  
    do {  
        q = (p + r) / 2;  
        if (e == v[q])  
            return q;  
        if (e < v[q])  
            r = q - 1;  
        else  
            p = q + 1;  
    } while (p <= r);  
    return -1;  
}
```

Recursividade de cauda

- Uma função apresenta recursividade de cauda se nenhuma operação é executada após o retorno da chamada recursiva, exceto retornar seu valor. Ou seja, a chamada recursiva é o último comando da função.
- Em geral, compiladores, que executam otimizações de código, substituem as funções que apresentam recursividade de cauda por uma versão não recursiva dessa função.

Usando um FOR

Recursividade de cauda

Exemplo

Exemplo (Fibonacci)

```
/* Implementação ruim */  
int fib( int n ){  
    if( n == 0 || n == 1 )  
        return 1;  
    return fib( n-1 ) + fib( n-2 );  
}
```

**Mudar algoritmos
recursivos para não
recursivos.
Se tiver recursividade de
cauda é possível fazer essa
mudança.**

Exemplo (Fibonacci)

```
/* Implementação ruim */  
int fib( int n ){  
    if( n == 0 || n == 1 )  
        return 1;  
    return fib( n-1 ) + fib( n-2 );  
}
```

Relação de Recorrência:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

$$T(1) = O(1)$$

$$T(0) = O(1)$$

Exemplo (Fibonacci)

```
/* Implementação ruim */  
int fib( int n ){  
    if( n == 0 || n == 1 )  
        return 1;  
    return fib( n-1 ) + fib( n-2 );  
}
```

Relação de Recorrência simplificada:

$$T(n) = 2T(n-1) + O(1)$$

$$T(0) = O(1)$$

Relação de Recorrência

$$T(n) = 2T(n-1) + O(1)$$

$$2T(n-1) = 2^2T(n-2) + 2O(1)$$

$$2^2T(n-2) = 2^3T(n-3) + 2^2O(1)$$

...

...

$$2^{n-1}T(1) = 2^nT(n-n) + 2^{n-1}O(1)$$

$$2^nT(0) = 2^n O(1)$$

$$T(n) = \sum_{i=0}^n 2^i = 2^{n+1} - 1$$

$$O(2^n)$$

Relação de Recorrência:

$$T(n) = 2T(n-1) + O(1)$$

$$T(0) = O(1)$$

Exemplo (Fibonacci)

Ao usar a relação de recorrência correta
chegaríamos em uma resposta parecida:

Relação de Recorrência:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

$$T(1) = O(1)$$

$$T(0) = O(1)$$

$$O(\varphi^n)$$

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

$$\varphi \cong 1,6180$$

Propriedades dos Somatórios

$$\sum_{i=0}^n ca_i = c \sum_{i=0}^n a_i \quad (\text{Distributiva})$$

$$\sum_{i=0}^n (a_i + b_i) = \sum_{i=0}^n a_i + \sum_{i=0}^n b_i \quad (\text{Associativa})$$

$$\sum_{i=n}^0 a_i = \sum_{i=0}^n a_i \quad (\text{Comutativa})$$

Propriedades dos Somatórios

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^n a^i = \frac{a^{n+1} - 1}{a - 1}$$

$$\sum_{i=1}^{\log n} i = n \log n$$

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

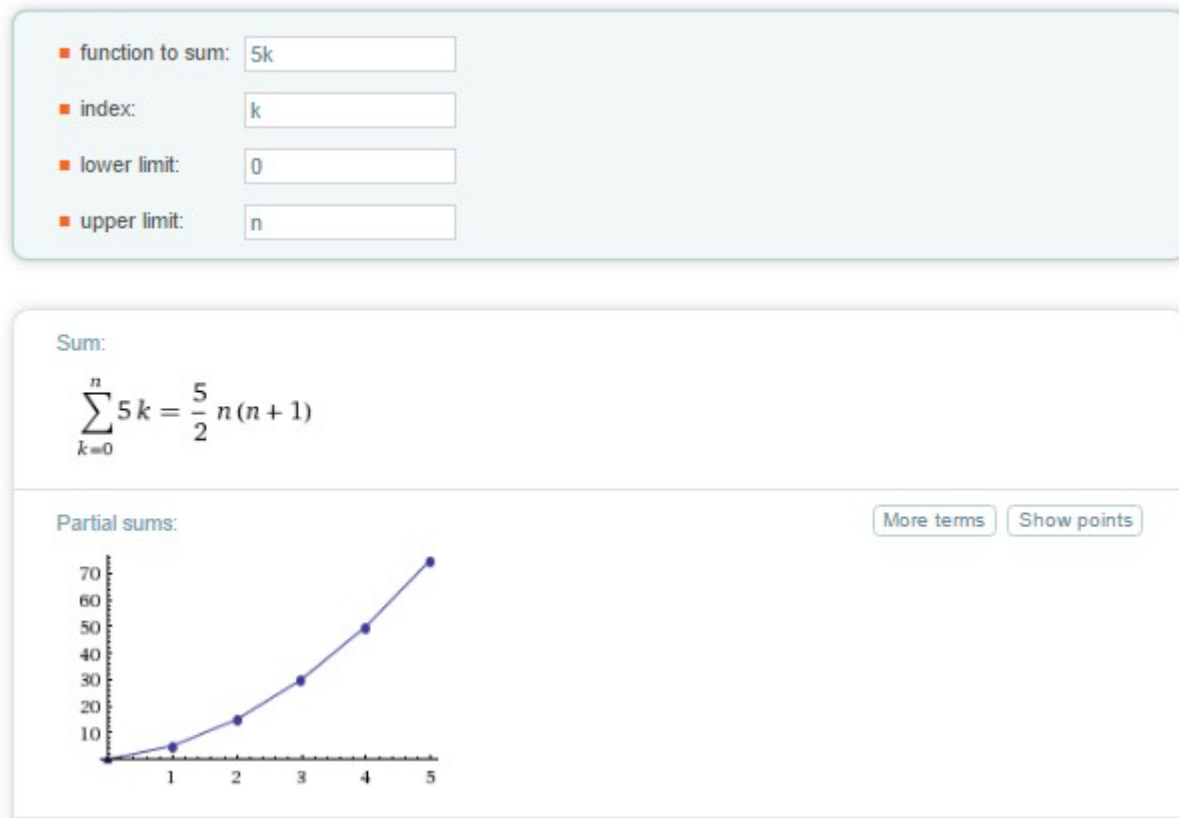
$$\sum_{i=0}^{\log n} 2^i = 2n - 1$$

$$\sum_{i=1}^n \log i = \log(n!)$$

$$\frac{1}{2}n \log n \leq \log(n!) \leq n \log n \quad \therefore \quad \log(n!) = \Theta(n \log n)$$

Somatórios

Existem algumas ferramentas que calculam as fórmulas dos somatórios e até mesmo geram gráficos.



Ex: <https://www.wolframalpha.com/input/?i=sum>

Propriedades das Potências

$$a^m a^n = a^{m+n}$$

$$\frac{a^m}{a^n} = a^{m-n}, a \neq 0$$

$$(ab)^n = a^n b^n$$

$$\left(\frac{a}{b}\right)^n = \frac{a^n}{b^n}, b \neq 0$$

$$(a^m)^n = a^{nm}$$

Propriedades dos Logaritmos

$$a^b = c \Leftrightarrow \log_a c = b$$

$$a^{\log_a b} = b$$

$$\log_c (ab) = \log_c a + \log_c b$$

$$\log_b a^c = c \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$a^{\log_b c} = c^{\log_b a}$$

Atividade 1

Escreva duas versões do algoritmo de fibonacci: a versão “ruim” apresentada e uma versão “boa” usando vetores. Faça uma comparação de tempo de execução com valores entre 25 e 60.

Qual a complexidade da versão “boa” do algoritmo?

Atividade 2

Elabore um algoritmo recursivo para calcular a raiz quadrada de um número real (positivo) qualquer. Após implementar o algoritmo, escreva a relação de recorrência e a respectiva complexidade do algoritmo para o pior caso.

*Lembrete: comparação de números reais através de aproximação $\Rightarrow |x - 3.2| \leq 0.00001$

*OBS: $r = \sqrt{n}$ quando $n \geq 1$, $1 \leq r \leq n$
 quando $n < 1$, $n \leq r \leq 1$

Atividade 3

Resolva as seguintes recorrência por substituição:

(Utilize os somatórios conhecidos para chegar a um resultado final ou utilize a ferramenta apresentada)

a. $T(n) = T(n-1) + \Theta(\log n)$

$$T(1) = \Theta(\log 1)$$

b. $T(n) = 3T(n-1) + \Theta(n)$

$$T(1) = \Theta(1)$$

c. $T(n) = T(\sqrt{n}) + \Theta(1)$

$$T(2) = \Theta(1)$$

Não precisa entregar, mais como desafio

Resolução de Recorrências (Substituição)

a) $T(n) = T(n-1) + \Theta(\log n)$

Expansão:

$$T(n) = T(1) + \sum \log i \text{ (de } i=2 \text{ até } n)$$

$$\sum \log i = \log(n!) = \Theta(n \log n)$$

Resultado: $T(n) = \Theta(n \log n)$

b) $T(n) = 3T(n-1) + \Theta(n)$

Expansão leva a:

$$T(n) = 3^{n-1}T(1) + \sum 3^j (n-j)$$

A soma converge para constante multiplicada por 3^n

Resultado: $T(n) = \Theta(3^n)$

c) $T(n) = T(\sqrt{n}) + \Theta(1)$

Após k expansões: $T(n) = T(n^{1/2^k}) + k$

Parada quando $n^{1/2^k} = 2 \rightarrow k = \Theta(\log \log n)$

Resultado: $T(n) = \Theta(\log \log n)$

Resolução de Recorrências por Substituição

a) $T(n) = T(n-1) + \Theta(\log n)$

Expansão por substituição:

$$T(n) = T(n-1) + \Theta(\log n)$$

$$T(n) = T(n-2) + \Theta(\log(n-1)) + \Theta(\log n)$$

...

$$T(n) = T(1) + \sum_{i=2}^n \Theta(\log i)$$

Como $T(1) = \Theta(1)$, temos:

$$T(n) = \Theta(1) + \Theta(\sum \log i)$$

$$\sum \log i = \log(n!)$$

$$\text{E } \log(n!) = \Theta(n \log n)$$

Resultado final: $T(n) = \Theta(n \log n)$

b) $T(n) = 3T(n-1) + \Theta(n)$

Expansão:

$$T(n) = 3T(n-1) + \Theta(n)$$

$$T(n) = 3^2 T(n-2) + 3\Theta(n-1) + \Theta(n)$$

$$T(n) = 3^3 T(n-3) + 3^2\Theta(n-2) + 3\Theta(n-1) + \Theta(n)$$

...

$$T(n) = 3^{n-1}T(1) + \sum_{j=0}^{n-2} 3^j \Theta(n-j)$$

Como $T(1) = \Theta(1)$:

$$T(n) = \Theta(3^{n-1}) + \sum \Theta(3^j (n-j))$$

A soma cresce proporcionalmente a 3^n

Resultado final: $T(n) = \Theta(3^n)$

c) $T(n) = T(\sqrt{n}) + \Theta(1)$

Expansão:

$$T(n) = T(n^{1/2}) + \Theta(1)$$

$$T(n) = T(n^{1/4}) + 2\Theta(1)$$

...

Após k passos: $T(n) = T(n^{1/2^k}) + k\Theta(1)$

Parada quando $n^{1/2^k} = 2$

Tomando log:

$$(1/2^k) \log n = \log 2$$

$$2^k = \Theta(\log n)$$

$$k = \Theta(\log \log n)$$

Resultado final: $T(n) = \Theta(\log \log n)$